

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Observation Task No.	2
Student Name	B.JATIN
Roll No.	A24126552070
Date	21-09-2026

1. TITLE

Node.js File Management Using File System Module

2. AIM

To develop a Node.js application that accepts a filename and content from the user and performs file creation, writing, reading, appending and final content display using the File System module.

3. OBJECTIVE

1. To use the built-in Node.js File System (fs) module.
2. To accept filename and content from the terminal.
3. To create or write a file.
4. To read and display file contents.
5. To append additional content to an existing file.
6. To read and display the final file contents.
7. To understand basic server-side file operations.

4. THEORY

Node.js provides the built-in File System module, commonly imported as fs, for working with files. `writeFileSync()` creates or overwrites a file, `readFileSync()` reads its contents, and `appendFileSync()` adds content without removing the existing data. The `readline` module is used here to collect input from the terminal.

Application Flow: Enter Filename → Enter Initial Content → Write File → Read File → Enter Additional Content → Append → Read Final File

5. ALGORITHM / PROCEDURE

1. Create a Node.js file named `file-management.js`.
2. Import the `fs` and `readline` modules.
3. Create a `readline` interface for terminal input.
4. Accept a filename and initial content from the user.
5. Use `writeFileSync()` to create or overwrite the file.
6. Use `readFileSync()` to display the initial contents.
7. Accept additional content and use `appendFileSync()`.
8. Read and display the final contents.
9. Run the program from a fresh terminal and verify the created file.

6. PROGRAM

```
1: const fs = require("fs");
2: const readline = require("readline");
3:
4: // Create a terminal input/output interface.
5: const rl = readline.createInterface({
6:   input: process.stdin,
7:   output: process.stdout
8: });
9:
10: rl.question("Enter filename: ", function(filename) {
11:
12:   rl.question("Enter initial content: ", function(content) {
13:
14:     // Create or overwrite the file.
15:     fs.writeFileSync(filename, content);
16:
17:     console.log("\nFile created successfully!");
18:
19:     // Read the file contents.
20:     let data = fs.readFileSync(filename, "utf8");
21:
22:     console.log("\nInitial file content:");
23:     console.log(data);
24:
25:     rl.question("\nEnter additional content: ", function(additional) {
26:
27:       // Append additional content to the file.
28:       fs.appendFileSync(filename, "\n" + additional);
29:
30:       console.log("\nContent appended successfully!");
31:
32:       // Read the final file contents.
33:       let finalData = fs.readFileSync(filename, "utf8");
34:
35:       console.log("\nFinal file content:");
36:       console.log(finalData);
37:
38:       rl.close();
39:     });
40:   });
41: });
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>require('fs')</code>	Loads Node.js's built-in File System module.
<code>readline.createInterface()</code>	Creates an interface for terminal input and output.
<code>writeFileSync()</code>	Creates or overwrites a file with the supplied content.
<code>readFileSync()</code>	Reads file contents synchronously.
<code>appendFileSync()</code>	Adds content to the end of an existing file.
<code>rl.close()</code>	Closes the readline interface.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Filename test.txt + initial content	File is created	File created successfully	Pass
TC-02	Read created file	Initial content displayed	Initial content displayed	Pass
TC-03	Enter additional content	Content appended	Content appended successfully	Pass
TC-04	Read final file	Both contents displayed	Final content displayed	Pass

9. OUTPUT

Terminal execution showing file creation, initial file reading and the prompt for additional content.

The screenshot shows the VS Code editor with the file-management.js file open. The code defines a readline interface for file management. The terminal output shows the execution of the script, which prompts for a filename, creates a file, reads its content, and prompts for additional content.

```

1  const fs = require('fs');
2  const readline = require('readline');
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });
8
9  rl.question("Enter filename: ", function (filename) {
10    rl.question("Enter initial content: ", function (content) {
11      // Create and write the file
12      fs.writeFileSync(filename, content);
13
14      console.log("\nFile created successfully!");
15
16      // Read the file
17      let data = fs.readFileSync(filename, "utf8");
18    });
19  });

```

```

at Module._load (node:internal/modules/cjs/loader:1396:12)
at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
at node:internal/main/run_main_module:33:47

Node.js v24.18.0
PS C:\Users\badag\OneDrive\Desktop\FS_LAB\observation\week5\file-management> node file-management.js
Enter filename:
Enter initial content: hello this is test.txt
Enter additional content:

```

10. RESULT

Thus, the Node.js File Management application was successfully developed using the fs and readline modules. The program creates, reads, appends and displays file contents through terminal interaction.